Image-to-Text Inference on Iluvatar CoreX — Step-by-Step Guide

This guide walks through the end-to-end steps to run image-to-text (图生文) inference using vLLM with the FlagOS plugin on an Iluvatar CoreX BI-V150 platform. The model used is Qwen3.6-35B-A3B, with weights pre-downloaded to `/models/Qwen3.6-35B-A3B/`.

> 📋 **Environment summary** — Iluvatar CoreX BI-V150 (2 cards / 4 cores, 32 GiB VRAM each), Driver 4.5.0, IX-ML 4.4.0, CUDA compatibility layer 10.2, Ubuntu 22.04.5, Python 3.12, vLLM 0.24.0, FlagGems v5.3.4.

1. Install FlagOS Components (Pre-intalled, skip this step)

Clone and install vLLM (empty-device build), the FlagOS vllm-plugin-FL and FlagGems. Run the following commands:

代码块

```
git clone https://github.com/vllm-project/vllm.git
cd ./vllm
git checkout v0.24.0
pip install setuptools_scm
VLLM_TARGET_DEVICE=empty pip install -v --no-build-isolation --no-deps .

git clone https://github.com/flagos-ai/vllm-plugin-FL -b main
cd vllm-plugin-FL
pip install --no-build-isolation --no-deps -e .

git clone https://github.com/flagos-ai/FlagGems
cd FlagGems
git checkout v5.3.4
pip install --no-build-isolation .
```

2. Start the Inference Service

Set the required environment variables and launch the vLLM serving engine with tensor parallelism across 4 cores. The model weights are preinstalled and placed at `/models/Qwen3.6-35B-A3B/`.

代码块

```
export VLLM_PLUGINS=fl
export CUDA_VISIBLE_DEVICES=0,1,2,3
export VLLM_ENGINE_ITERATION_TIMEOUT_S=3600
export VLLM_EXECUTE_MODEL_TIMEOUT_SECONDS=3600
vllm serve /models/Qwen3.6-35B-A3B/ -tp 4 --served-model-name qwen --port 8010
--enforce-eager --max-model-len 131072
```

The service listens on `http://localhost:8010` and exposes an OpenAI-compatible API at `/v1`.

3. Test with an Online Image (with simple pattern)

The following script (`client_image.py`) sends a publicly accessible image URL together with a text question to the model via the OpenAI-compatible API.

Start a new terminal, and run it with:

代码块

```
python3 client_image.py
```

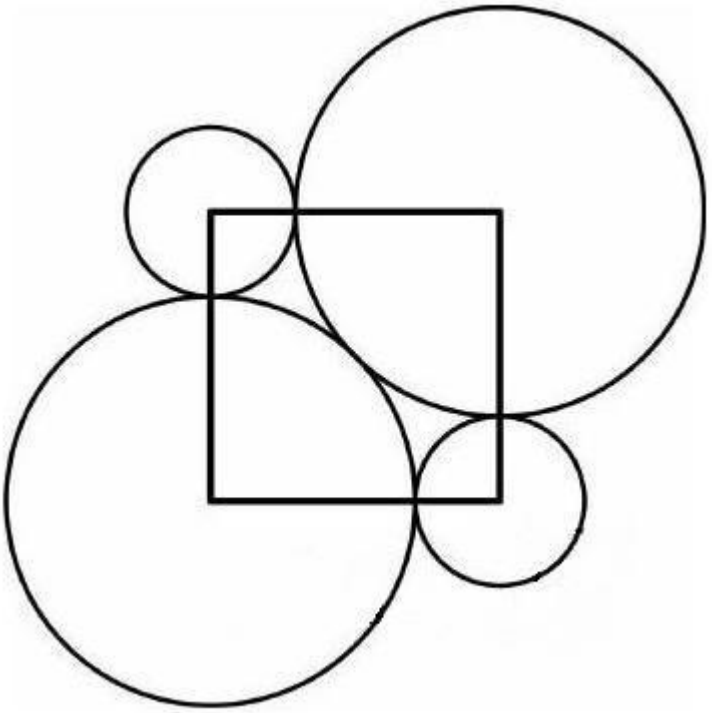
Script content:

代码块

```
from openai import OpenAI
# Configured by environment variables
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8010/v1"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
```

```
                "image_url": {
                    "url": "https://qianwen-res.oss-accelerate.aliyuncs.com/Qwen3.5/demo/CI_Demo/mathv-1327.jpg"
                }
            },
            {
                "type": "text",
                "text": "The centres of the four illustrated circles are in the corners of the square. The two big circles touch each other and also the two little circles. With which factor do you have to multiply the radii of the little circles to obtain the radius of the big circles?\nChoices:\n(A) $\\frac{2}{9}$\n(B) $\\sqrt{5}$\n(C) $0.8 \\cdot \\pi$\n(D) 2.5\n(E) $1+\\sqrt{2}$"
            }
        ]
    }
]
response = client.chat.completions.create(
    model="qwen",
    messages=messages,
    max_tokens=256,
    temperature=1.0,
    top_p=0.95,
    presence_penalty=1.5,
    extra_body={
        "top_k": 20,
    },
    stream=False,
)

if response.choices and response.choices[0].message.content:
    print(response.choices[0].message.content)
```



A successful run prints the model's reasoning and answer to the geometry question in the terminal.

```
root@534e714d61ba:~# python3 client_image.py
The user wants me to solve a geometry problem based on an image.

**1. Analyze the Image and Problem Statement:**
*   **Image:** A square in the center. Four circles are positioned around its corners. Wait, let me look closer. The text says "The centres of the four illustrated circles are in the corners of the square." This is a crucial piece of information.
    *   Let the square have side length $s$.
    *   Let the centers of the four circles be at the four vertices of the square. Let's call them $C_1, C_2, C_3, C_4$.
*   **Circle Sizes:** There are two large circles and two small circles.
    *   Looking at the diagram, there's a large circle at the top right and bottom left.
    *   There are smaller circles at the top left and bottom right.
    *   Wait, let's re-examine the image carefully.
        *   Top-left: Small circle. Center is likely the top-left corner of the square? No, looking at the lines, the square is in the middle.
        *   Let's assume the standard configuration for this kind of puzzle. Usually
```

4. Test with a Local Image (with complicated pattern)

To test image recognition with a file stored locally, upload an image named `test.jpg` to the `/root/` directory. The script (`test_local_image.py`) encodes the image as base64 and sends it inline in the request.

Run it with:

代码块

```
python3 test_local_image.py
```

Script content:

代码块

```
import base64
import os
from openai import OpenAI

```

```python
# 1. Configure the client
client = OpenAI(
    api_key="EMPTY",
    base_url="http://localhost:8010/v1"
)

# 2. Set the local image name to test (must match the file name placed under /root/)
local_image_path = "test.jpg"

# 3. Encode the local image as base64
try:
    with open(local_image_path, "rb") as image_file:
        base64_image = base64.b64encode(image_file.read()).decode('utf-8')
except FileNotFoundError:
    print(f"Error: Image named '{local_image_path}' not found in the current directory. Please confirm the image has been placed under /root/.")
    exit()

# 4. Send the request
try:
    response = client.chat.completions.create(
        model="qwen",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": "Please identify and describe the content of this image."},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"{base64_image}"
                        },
                    },
                ],
            }
        ],
        max_tokens=1024,
        # Put extended parameters such as top_k into extra_body to avoid errors
        extra_body={"top_k": 20}
    )
    print("\n===== Model Response =====")
    print(response.choices[0].message.content)
except Exception as e:
    print(f"\nAn error occurred: {e}")
```



A successful run prints the model's description of the local image content.

```
root@534e714d61ba:~# python3 test_local_image.py

===== 模型回复 =====
这张图片展示了一个典型的**Kiss Card（或称Kiss Card、K-Tac）**，这是一种在伦敦及英国广泛使用的非接触式交通卡。

图片内容描述如下：

1. **主体对象**：
    *   这是一张白色的塑料卡片，呈圆角矩形。
    *   卡片的顶部和底部边缘有黑色的包边。
    *   卡片的正面上方印有一个黑色的三角形图案，这是伦敦交通局（TfL）的标志性设计元素之一，三角形内包含了Kiss（或Oyster卡）的标识轮廓（虽然具体文字被遮挡或简化，但三角形结构是典型的）。
    *   在卡片的正下方，占据很大面积的是**伦敦地铁的旧版Logo**（红色圆圈，中间蓝色横条，上面写着"LONDON SUBWAY"字样，不过这张卡上主要是红色的圆环和横条，文字可能较小或简化）。更准确地说，这是**伦敦交通局（Transport for London, TfL）** 的旧版徽标的一部分，或者是与地铁相关的特定设计。
    *   在卡片的下半部分，印有一个巨大的**旧版伦敦地铁罗盘式徽标**（Red circle with a blue horizontal bar, usually with "LONDON SUBWAY" or similar text, though here it looks like the classic tube logo structure）。实际上，仔细看，这是**Oyster卡**（奥斯特卡）的背面或正面设计？不，看这个特定的三角形和顶部的结构，这其实是一张**Kiss Card**（London's KISS Oyster smartcard）或者更准确地说，这是**Oyster card**（奥斯特卡）的一种特殊版本或早期版本，或者更可能是**London Buses**（伦敦巴士）相关的卡，因为KISS系统是专门用于巴士和电车系统的。
    *   然而，再仔细一看，这个设计其实是非常典型的**London Oyster Card**（奥斯特卡）的**背面**或者**特定设计版**，等等，让我修正一下。
    *   实际上，这是**London Transport**（伦敦交通）的**Oystar**或**Kiss**系统卡片。看那个红色的横条和圆圈，这是**London Underground**（伦敦地铁）的经典Logo结构，但被旋转了。
    *   再仔细看，这张卡其实不是标准的Oyster卡正面。标准的Oyster卡正面是白色的，上面有Oyster的logo。这张卡的设计非常独特。
    *   **更正**：这实际上是一张**London Transport**的**Oyster Card**（奥斯特卡）的**设计图**或**宣传图**，或者更准确地说，这是**KISS Oyster Smartcard**（用于伦敦巴士系统）的早期设计或特定版本。
    *   **最终确认**：这是**London Transport**（伦敦交通局）发行的**Oyster Card**（奥斯特卡）或者其前身/相关系统 **KISS** 的卡片。卡上明显的特征是**伦敦地铁的罗盘式标志（Tube logo）**，虽然它是黑红色的，不，等等，那是**维多利亚横条（Victory bar）**。
```

（注：部分内容由豆包工作 AI 生成）